PROJECT DOCUMENTATION

An AI-Based Personal Finance Advisor Bot

Dr.D.Y. Patil College of Engineering and Innovation

Department of Computer Engineering

Academic Year 2026–27

Submitted By:

Vedika Rajendra Karlekar

PERSONAL FINANCE ADVISOR BOT: PROJECT SCOPING REPORT

Phases, dependencies, risks, and delivery plan. Prepared as a working draft for planning and stakeholder review.

Document status: Draft v1.0 for review **Basis:** Assumptions listed in Section 2.4. Replace them with your actual constraints and the plan will tighten quickly.

1. Executive Summary

This report scopes the design and delivery of a Personal Finance Advisor Bot: a conversational assistant that helps individuals understand their money, build budgets, set and track goals, reduce debt, and make better-informed everyday financial decisions. The bot connects to a user's financial data (with consent), answers questions in plain language, produces personalized insights, and proactively nudges users toward their goals.

The product sits at an unusual intersection. It needs the fluency of a modern large language model, the numerical exactness of a calculator, the data plumbing of a fintech aggregator, and the caution of a regulated advisory service. Most of the project risk lives in the seams between these four, not inside any single one.

The central scoping recommendation is to **separate what the bot says from what the bot computes and what the bot is allowed to recommend.** Language generation should never be the source of a number. Every figure shown to a user should come from a deterministic calculation engine or a verified data source, and every recommendation should pass through a policy layer that enforces the product's regulatory posture. This one architectural decision removes the largest category of risk (confident but wrong financial answers) and makes the remaining work tractable.

Headline plan

- **Duration:** about 9 months from kickoff to general availability, delivered in six phases with a limited beta at month 6.
- **Team:** a core team of roughly 10 to 12 people (product, design, mobile and web engineering, backend, data and ML, security and compliance, QA, and a part-time legal advisor).
- **Product posture:** educational and informational guidance in the first release (budgeting, cash flow, debt payoff, goal tracking). Personalized investment advice is deliberately deferred because it triggers the heaviest regulatory obligations in most jurisdictions.
- **Top five risks:** incorrect or misleading financial guidance, regulatory classification as an adviser, data aggregation reliability, security and privacy incidents, and user trust and adoption.
- **Key dependencies:** a bank data aggregation provider, an LLM provider with acceptable data handling terms, legal sign-off on product claims, and a cloud environment that meets the security baseline.

The remaining sections define goals and metrics, scope, users, requirements, architecture, data and integrations, AI design, compliance, the phased plan, timeline and resourcing, dependencies, a full risk register, testing, rollout, operations, and open questions.

2. Vision, Goals, and Success Metrics

2.1 Vision

Make sound financial guidance as accessible as a text message. Most people do not have a financial planner, and many of the decisions that shape their finances (how much to save, which debt to pay first, whether a purchase fits the month) are small, frequent, and poorly supported by existing tools. A bank app shows balances. A spreadsheet needs expertise. The bot closes the gap by combining a person's actual data with clear explanations and practical next steps.

2.2 Product goals

1. **Clarity:** users understand where their money goes within their first session.
2. **Control:** users create and keep a budget or savings plan with less effort than manual tools require.
3. **Confidence:** users can ask a question in everyday language and get an answer they can trust and act on.
4. **Progress:** users measurably improve at least one financial outcome, such as savings rate, debt balance, or overdraft frequency

2.3 Success metrics

Metrics should be defined before design begins so that scope decisions can be tested against them. The table below proposes a starting set. Targets are initial hypotheses and should be calibrated during the beta.

Category	Metric	Initial target (beta)	Notes
Activation	Users who connect at least one account within 24 hours of signup	55%	Strong predictor of retention
Engagement	Weekly active users as share of monthly active users	40%	Indicates habit formation
Quality	Answers rated helpful by users	85%	Thumbs rating plus sampled review
Accuracy	Numerical answers matching the calculation engine	100%	Non-negotiable; failures are release blockers
Safety	Policy violations per 1,000 responses	Below 1	Measured by automated checks and audits
Outcome	Users with an active goal who improve savings rate in 90 days	30%	Primary value metric
Reliability	Chat response time at the 95th percentile	Under 6 seconds	Includes data retrieval

Category	Metric	Initial target (beta)	Notes
Trust	Users who report feeling in control of their finances	Up 15 points vs baseline	In-app survey
Retention	90-day retention	35%	Industry-typical range for finance apps is lower, so this is ambitious

Two metrics deserve special attention. **Accuracy** is binary in nature: a financial assistant that is right 98% of the time is still wrong for roughly one in fifty users, and some of those errors will involve real money. **Outcome** is the only metric that proves the product works rather than merely being used.

2.4 Planning assumptions

Because no project details were supplied, this report is built on the following assumptions. Each one materially affects the plan.

- **Audience:** retail consumers, adults, individual rather than joint accounts in the first release.
- **Channels:** a mobile app (iOS and Android) and a responsive web app, with the chat experience as the core interface.
- **Data sources:** bank, card, loan, and investment account data through a third-party aggregation provider, plus manual entry and CSV or statement upload as a fallback.
- **Posture:** informational and educational guidance, not regulated personalized investment advice, at launch.
- **Model approach:** a hosted large language model accessed through an API, orchestrated with tools and a deterministic calculation engine.
- **Jurisdictions:** one primary launch market, with architecture that allows later expansion. Regulatory details are therefore described generically, with examples where useful.
- **Timeline and team:** about nine months, with the team sizes given in Section 11.
- **Funding:** sufficient to cover the team, cloud, aggregator and model costs, security testing, and legal review.

3. Scope Definition

A crisp scope boundary is the most effective risk control available. Finance products attract scope creep because every user has a different situation and every stakeholder has a favorite feature.

3.1 In scope for release 1

- Account onboarding, identity verification as required by the aggregator, and consent management.
- Account connection through an aggregation provider, plus manual entry and file upload.
- Transaction ingestion, categorization, and merchant normalization.
- A conversational interface supporting questions about spending, income, balances, subscriptions, upcoming bills, and trends.
- Budget creation and tracking, including suggested budgets based on history.
- Savings goals with projected completion dates and recommended contributions.

- Debt payoff planning using standard strategies (avalanche, snowball) with transparent math.
- Cash flow forecasting for the next 30 to 90 days.
- Proactive notifications: unusual spending, upcoming bills, goal progress, low balance risk.
- Explainability: every number can be traced to its source transactions or formula.
- Privacy controls: data export, account disconnection, and full deletion.
- Administrative tooling for support, content review, and incident response.

3.2 Explicitly out of scope for release 1

- Personalized investment recommendations, security selection, or portfolio rebalancing instructions.
- Tax filing, tax optimization, or legal advice (general educational content only).
- Moving money: payments, transfers, or account opening on the user's behalf.
- Credit products, loan origination, or referral monetization that could create conflicts of interest.
- Joint accounts, business finances, and multi-currency support.
- Voice interface and third-party chat platform integrations.
- Human advisor handoff (planned for a later release, with the architecture left ready for it).

3.3 Deferred candidates for later releases

These are valuable but should not enter release 1: investment education and goal-based portfolio guidance (after regulatory review), tax-aware planning, household and partner sharing, a human advisor marketplace, credit score improvement coaching, and automated savings rules that move money.

3.4 Scope control rules

1. Any addition must name the metric it improves and the work it displaces.
2. Anything that moves money or gives investment-specific direction requires explicit legal sign-off before entering the plan.
3. A change that alters the data the bot stores or shares requires a privacy review.
4. Scope is reviewed at each phase gate, not continuously.

4. Users and Use Cases

4.1 Primary personas

The overwhelmed earner. Steady income, several accounts and cards, no clear picture of where money goes. Wants a fast diagnosis and a simple plan. Needs low-effort setup and plain language.

The debt reducer. Carrying credit card or loan balances. Wants a payoff plan and reassurance that it is working. Needs transparent math and gentle, non-judgmental tone.

The goal saver. Saving for a specific outcome such as a home deposit, emergency fund, or trip. Wants to know if they are on track and what to change. Needs projections and scenario comparison.

The variable-income worker. Freelancer or gig worker with irregular earnings. Wants budgeting that adapts to uneven months. Needs forecasting that handles volatility.

4.2 Core user journeys

1. **First session:** sign up, connect an account, see a spending summary and three insights, set one goal. The aim is value within ten minutes.
2. **Ask a question:** type a question such as how much did I spend on food delivery last quarter, receive a precise answer with a breakdown and a source link.

3. **Plan a goal:** state a target and date, review a feasibility check, adjust the amount or date, and accept a recommended plan.
4. **Handle a shortfall:** receive a warning that the account is likely to run low before payday, with options for reducing or deferring spending.
5. **Pay down debt:** compare payoff strategies, see interest saved, and track progress month by month.
6. **Control data:** disconnect an account, export history, or delete everything, with clear confirmation.

4.3 Representative questions the bot must handle

- Descriptive: What did I spend on groceries last month, and how does that compare to my average?
- Diagnostic: Why was my spending higher in March?
- Predictive: Will I have enough to cover rent and bills before my next paycheck?
- Prescriptive (within scope): How much should I put toward my emergency fund each month to reach three months of expenses by December?
- Educational: What is the difference between the avalanche and snowball methods?
- Out-of-scope handling: Should I buy this particular stock? The bot declines the specific recommendation, explains why, and offers general education or a pointer to a qualified professional.

5. Requirements

5.1 Functional requirements

ID	Requirement	Priority
F1	Authenticate users with strong authentication and support biometric unlock on mobile	Must
F2	Connect, refresh, and disconnect financial accounts through the aggregator	Must
F3	Ingest and categorize transactions with user-correctable categories that the system learns from	Must
F4	Answer natural-language questions using verified data and the calculation engine	Must
F5	Create, edit, and track budgets by category and period	Must
F6	Create and track goals with projections	Must
F7	Generate debt payoff plans with interest and timeline calculations	Must
F8	Forecast cash flow for 30 to 90 days with an uncertainty range	Should
F9	Send proactive alerts with user-controlled frequency and channels	Should
F10	Show provenance for every figure (source accounts, transactions, formula)	Must
F11	Detect out-of-scope or risky requests and respond with approved language	Must
F12	Support data export and complete deletion on request	Must

ID	Requirement	Priority
F13	Provide feedback controls on every answer and a way to report problems	Must
F14	Offer manual entry and file upload when aggregation is unavailable	Should
F15	Support accessibility standards (screen readers, scalable text, contrast)	Must
F16	Provide an internal console for support, audit, and content management	Should

5.2 Non-functional requirements

- **Accuracy:** all monetary values come from deterministic computation; regression suites verify this on every build.
- **Performance:** typical question answered in under 6 seconds at the 95th percentile; first token streamed within 2 seconds.
- **Availability:** 99.9% monthly availability for the core chat and data services; graceful degradation when the aggregator or model provider is down.
- **Scalability:** designed for 10 times the beta load without architectural change.
- **Security:** encryption in transit and at rest, least privilege access, secrets management, audited access to sensitive data, and independent penetration testing before launch.
- **Privacy:** data minimization, purpose limitation, clear consent records, regional data residency where required, and no use of customer financial data to train third-party models.
- **Auditability:** every response can be reconstructed (inputs, tool calls, policy decisions, model version) for dispute resolution and regulatory inquiry.
- **Explainability:** users can see why a recommendation was made in plain language.
- **Localization readiness:** currency, date, and locale handling designed in from the start, even though release 1 ships with one market.
- **Accessibility:** conformance to WCAG 2.2 level AA for the web experience and equivalent platform guidelines on mobile.

5.3 Quality attributes ranked

When requirements conflict, the team needs a tiebreaker. The proposed ordering is: **safety and correctness, then privacy and security, then trust and clarity, then speed, then breadth of features.** A slower correct answer is better than a fast wrong one, and a smaller feature set that users trust beats a larger one they doubt.

6. Solution Architecture

6.1 Guiding principles

1. **The model does not do math and does not hold facts.** It interprets intent, selects tools, and phrases results. Numbers come from tools.
2. **Policy sits outside the model.** Rules about what the bot may recommend are enforced in code and configuration, not only in prompts.
3. **Everything is traceable.** Each answer carries references to the data and calculations behind it.
4. **Fail safe.** When data is stale, a tool fails, or confidence is low, the bot says so and offers a fallback instead of guessing.
5. **Keep sensitive data narrow.** The model sees only the minimum data needed for the current question, preferably aggregated or tokenized.

6.2 Logical components

Component	Responsibility
Client apps (iOS, Android, web)	Chat UI, dashboards, onboarding, notifications, consent screens
API gateway and auth	Authentication, rate limiting, request routing, session management
Conversation orchestrator	Manages dialogue state, prompts, tool selection, and response assembly
Intent and safety layer	Classifies requests, detects out-of-scope or risky topics, applies input filters
LLM gateway	Abstracts model providers, handles retries, fallbacks, logging, and redaction
Calculation engine	Deterministic budgeting, goal, debt, interest, and forecasting functions with unit tests
Financial data service	Normalized store of accounts, balances, and transactions with freshness metadata
Aggregation connector	Integrates the aggregator, handles consent, refresh, and error states
Categorization service	Merchant normalization and category assignment, with learning from user corrections
Insights and rules engine	Detects patterns, anomalies, bills, subscriptions, and triggers notifications
Policy and compliance layer	Output checks, required disclosures, prohibited recommendations, audit logging
Knowledge base	Curated, versioned educational content used for retrieval
Notification service	Push, email, and in-app messages with preference management
Admin console	Support tools, audit search, content and rule management, incident controls
Observability stack	Logs, metrics, traces, evaluation dashboards, and alerting

6.3 Request flow for a typical question

1. The user submits a question. The gateway authenticates and applies rate limits.
2. The safety layer classifies the request. Out-of-scope topics short-circuit to approved responses.
3. The orchestrator asks the model to identify intent and required tools.

4. Tools execute against the financial data service and calculation engine. Results include values, units, time ranges, and data freshness.
5. The model composes an explanation using only tool outputs and approved knowledge snippets.
6. The policy layer verifies that every number in the draft matches a tool output, that required disclosures are present, and that no prohibited recommendation appears.
7. The response streams to the user with provenance links. The full trace is stored for audit.
8. Feedback signals flow to the evaluation pipeline.

Step 6 is the control that most directly protects users. A draft that contains a number not found in tool output is blocked or regenerated, never shown.

6.4 Key architectural decisions to make early

Decision	Options	Recommendation
Model strategy	Single hosted model; multiple providers; self-hosted	Hosted primary model behind a gateway with a second provider for fallback
Orchestration	Custom code; agent framework	Thin custom orchestrator with explicit tool contracts, to keep behavior predictable and testable
Data store	Relational; document; hybrid	Relational core for ledgers and goals, with a vector index only for educational content
Categorization	Rules only; ML; LLM-assisted	Rules and ML first, LLM assistance for ambiguous merchants, with user corrections as ground truth
Hosting	Single cloud; multi-cloud	Single cloud with infrastructure as code and region flexibility
Client approach	Native; cross-platform	Cross-platform mobile framework plus responsive web, unless the team has strong native skills
Memory of user context	Full history in prompt; structured summaries	Structured profile and goal state, with retrieval of recent history per question

These decisions should be recorded as short architecture decision records during Phase 2 so that later changes are deliberate.

7. Data and Integrations

7.1 Data domains

Domain	Examples	Sensitivity	Source
Identity	Name, email, phone, verification status	High	User, identity provider

Domain	Examples	Sensitivity	Source
Accounts	Institution, account type, masked number, balance, currency	High	Aggregator, manual entry
Transactions	Date, amount, merchant, category, pending status	High	Aggregator, uploads
Goals and budgets	Targets, dates, category limits, contributions	Medium	User, bot suggestions
Conversation	Messages, tool calls, feedback ratings	High	User, system
Derived insights	Recurring bills, subscriptions, anomalies, forecasts	Medium	Insights engine
Consent and audit	Consent records, access logs, policy decisions	High	System
Educational content	Articles, definitions, worked examples	Low	Editorial team

7.2 Aggregation provider

The aggregator is the single most important external dependency. It determines which institutions can be connected, how fresh the data is, how often connections break, and how much consent and compliance burden the product carries.

Selection criteria

- Coverage of the target market's banks, cards, loans, and investment providers, measured by the share of the target users' institutions supported.
- Connection reliability: success rate, mean time to repair broken connections, and transparency about outages.
- Data quality: transaction history depth, enrichment (merchant names, categories), and handling of pending-to-posted changes.
- Regulatory posture: licensing or registration, support for official open banking or account aggregator frameworks where they exist, and consent management features.
- Security: certifications, tokenized credentials, and breach history.
- Commercial terms: per-user or per-call pricing, minimums, termination rights, and data ownership terms.
- Developer experience: sandbox quality, webhooks, documentation, and support responsiveness.

Run a two-vendor bake-off in Phase 1 using real test accounts. Abstract the integration behind a connector interface so that a second provider can be added without touching downstream services.

7.3 Data quality challenges to plan for

1. **Duplicate and split transactions** when providers re-send or revise records.
2. **Pending versus posted** states that change amounts or dates.
3. **Merchant ambiguity**, where one descriptor maps to several merchants or one merchant has many descriptors.
4. **Transfers between a user's own accounts** that must not be counted as spending or income.

5. **Refunds, reversals, and chargebacks** that distort category totals.
6. **Stale connections** where the user has not reauthorized and balances silently age.
7. **Currency and timezone handling** for dates near period boundaries.
8. **Incomplete history** when an account is newly connected.

Each of these should have an explicit handling rule, a test suite, and a user-facing explanation. The bot should state data freshness whenever it matters, for example: this answer uses data last updated this morning.

7.4 Categorization strategy

Categorization quality drives almost every insight, so it merits its own workstream. The approach is layered: deterministic rules for known merchants, an ML classifier for the long tail, LLM assistance for genuinely ambiguous cases, and user corrections treated as authoritative and applied to similar future transactions. Track category accuracy against a hand-labeled benchmark and report it with each release. A reasonable target is 92% or better accuracy on the top 20 categories by spend before beta.

7.5 Other integrations

Integration	Purpose	Notes
Identity and verification	Confirm user identity where required	Often bundled with the aggregator or a separate KYC vendor
LLM provider	Language understanding and generation	Zero data retention terms; regional processing if needed
Push and email providers	Notifications	Preference center and unsubscribe handling required
Analytics	Product metrics	Strictly privacy-reviewed, with no raw financial data
Error and performance monitoring	Reliability	Scrub sensitive fields before sending
Payment provider	Subscription billing, if the product is paid	Out of scope if release 1 is free
Support tooling	Ticketing and chat	Role-based access with masked data by default

8. AI Design and Guardrails

8.1 What the model is and is not responsible for

The model does	The model does not
Interpret what the user is asking	Perform arithmetic or projections
Choose which tools to call	Recall balances or transactions from memory
Explain results in plain, empathetic language	Invent products, rates, laws, or institutions

The model does	The model does not
Ask a clarifying question when needed	Recommend specific securities or tax positions
Summarize and compare options	Override policy rules or disclosures

8.2 Tool contracts

Every tool exposed to the model has a strict schema, validated inputs, typed outputs, and documented failure behavior. Representative tools include:

- **get_transactions:** filters by date range, category, merchant, and account; returns records with provenance identifiers.
- **summarize_spending:** aggregates by category and period; returns totals, averages, and comparisons.
- **get_balances:** returns current balances with a freshness timestamp.
- **project_goal:** returns required monthly contribution or projected completion date for a goal, with assumptions listed.
- **plan_debt_payoff:** returns schedules under chosen strategies, total interest, and payoff dates.
- **forecast_cash_flow:** returns expected balance paths with a range, using recurring items and historical variability.
- **find_recurring:** identifies subscriptions and recurring bills with confidence levels.
- **search_knowledge:** retrieves approved educational snippets with version identifiers.

Tool outputs carry the numbers. The response composer is instructed to quote them exactly, and the policy layer verifies that it did.

8.3 Guardrail layers

1. **Input screening:** detect prompt injection, attempts to extract other users' data, abusive content, and distress signals.
2. **Scope classification:** route investment-specific, tax-specific, legal, or crisis topics to approved handling.
3. **Grounding enforcement:** require that figures, dates, and named entities in the output match tool results or approved content.
4. **Policy rules:** prohibited phrases and claims (guarantees, certainty about future returns, pressure language), required disclosures, and tone constraints.
5. **Confidence and freshness checks:** attach caveats when data is stale, history is short, or classification confidence is low.
6. **Output screening:** final scan for personal data leakage and unsafe content.
7. **Human review loop:** sampled transcripts reviewed weekly by a qualified reviewer, with fast escalation for serious issues.

8.4 Handling sensitive situations

Financial conversations often involve stress. The bot needs defined behavior for these cases:

- **Financial distress:** respond with empathy, avoid judgment, offer practical steps, and surface hardship resources and, where appropriate, credit counseling information.
- **Signs of crisis or self-harm:** stop the finance workflow, respond with care, and provide crisis support information for the user's region.

- **Suspected fraud or scams:** provide general protective steps and direct the user to their bank or the relevant authority; never ask for credentials.
- **Vulnerable users:** if signals suggest confusion or coercion, slow down, simplify, and encourage speaking with a trusted person or professional.
- **Requests for guarantees or get-rich advice:** decline gently and explain why no one can promise outcomes.

8.5 Prompt and behavior management

Treat prompts, policies, and tool descriptions as versioned code. Changes go through review, run against the evaluation suite, and are released behind flags. Keep a changelog linking each behavior change to the issue that motivated it.

8.6 Evaluation framework

Evaluation is a first-class workstream, not a late-stage test. It has four layers.

Layer	What it checks	Method	Cadence
Unit	Calculation engine correctness	Deterministic tests against known cases and independent reference implementations	Every commit
Tool use	Correct tool selection and parameters	Scripted scenarios with expected tool calls	Every commit
Response quality	Helpfulness, clarity, tone, and groundedness	Curated question set graded by rubric, model-assisted grading calibrated against human reviewers	Every prompt or model change
Safety and policy	Refusals, disclosures, injection resistance, distress handling	Red-team suite and adversarial prompts	Every release

Build the evaluation set from the persona journeys in Section 4, from anonymized beta questions, and from deliberately adversarial cases. Start with at least 300 scenarios before beta and grow it continuously. Each production incident adds a regression case.

8.7 Model change management

Model providers update and retire models. The LLM gateway should pin versions, allow canary routing of a small share of traffic to a candidate model, and automatically compare candidate and incumbent results on the evaluation set before promotion. Maintain a tested fallback model so that a provider outage or deprecation does not become a product outage.

9. Security, Privacy, and Compliance

9.1 Regulatory posture

The central compliance question is whether the product is giving regulated financial advice. Many jurisdictions regulate personalized recommendations about investments, insurance, and credit, and some regulate how financial data may be accessed and shared. Rules differ widely. Examples of regimes the team may need to consider, depending on launch market, include investment adviser regulations,

consumer financial protection and unfair or deceptive practices law, open banking or account aggregator frameworks, data protection laws such as GDPR or national equivalents, and sector security standards.

This report is not legal advice. The recommendation is to engage counsel in Phase 1, obtain a written opinion on the product's classification, and design the release 1 scope to stay within it. Counsel should review all marketing claims, in-app disclosures, terms, and the policy rule set before launch.

9.2 Compliance workstreams

- **Product classification and disclosures:** what the bot may say, how it describes itself, and required notices.
- **Consumer protection:** fair, clear, and not misleading communications; accessible complaint handling.
- **Data protection:** lawful basis and consent, purpose limitation, retention limits, subject rights (access, correction, deletion, portability), breach notification, and cross-border transfer rules.
- **Financial data access:** consent records, scope limits, revocation, and any requirements on the aggregator relationship.
- **AI-specific obligations:** transparency that the user is talking to an AI, and any emerging rules on automated decision support in the launch market.
- **Record keeping:** retention of conversations, policies, and decisions to the extent required, balanced against data minimization.
- **Accessibility and anti-discrimination:** ensure the product does not produce differential treatment on protected characteristics.

9.3 Security architecture

Area	Controls
Identity and access	Strong authentication, biometric unlock, session management, step-up checks for sensitive actions
Data protection	Encryption in transit and at rest, field-level encryption for account identifiers, key management with rotation
Secrets	Central secrets manager, no secrets in code or logs
Network	Private networking, web application firewall, rate limiting, DDoS protection
Application	Secure coding standards, dependency scanning, static and dynamic analysis, SBOM tracking
LLM specific	Prompt injection defenses, tool permission scoping, output filtering, per-user data isolation, no cross-user context
Logging	Structured audit logs, sensitive fields redacted, tamper-evident storage
Insider risk	Least privilege, just-in-time access, masked data by default in support tools, access reviews

Area	Controls
Resilience	Backups, tested restores, multi-zone deployment, disaster recovery runbooks
Assurance	Threat modeling in design, independent penetration test before beta and before GA, vulnerability disclosure program

9.4 Privacy by design

1. **Minimize:** collect only what features require. Do not store aggregator credentials; rely on tokens.
2. **Isolate:** strict tenant and user isolation in data stores, caches, and retrieval indexes.
3. **Limit model exposure:** send the model aggregated or masked data when possible; avoid account numbers and full names; use contracts that forbid training on customer data and require minimal retention.
4. **Be transparent:** a plain-language explanation of what is collected, why, and for how long, shown at the moment of consent.
5. **Give control:** one-tap disconnect, export, and deletion, with deletion propagating to backups on a defined schedule and to processors.
6. **Retain sparingly:** default retention periods for conversations and transactions, configurable by market.

9.5 Incident readiness

Prepare an incident response plan before beta: severity definitions, on-call rotation, communication templates, regulator and user notification procedures, and a rehearsed tabletop exercise. Include AI-specific incidents, such as a harmful response pattern discovered in production, with a kill switch that can disable a feature or revert to a safe fallback within minutes.

10. Phased Delivery Plan

The plan has six phases. Each ends with a gate: a short review against exit criteria that decides whether to proceed, adjust, or stop. Phases overlap modestly where dependencies allow, but no phase starts building on unvalidated assumptions from the one before.

10.1 Phase overview

Phase	Name	Duration	Key outcome
0	Mobilize	Weeks 1 to 2	Team, governance, and workspace ready
1	Discovery and feasibility	Weeks 3 to 8	Validated scope, vendor choices, legal opinion, risk spikes complete
2	Foundation and design	Weeks 9 to 14	Architecture, data model, design system, evaluation harness
3	Core build (thin slice, then increments)	Weeks 15 to 28	Working product for internal users
4	Hardening and closed beta	Weeks 29 to 36	Security, performance, and safety proven with real users

Phase	Name	Duration	Key outcome
5	Launch and stabilization	Weeks 37 to 40 and beyond	General availability and operational maturity

10.2 Phase 0: Mobilize (weeks 1 to 2)

Objectives: establish ownership, working agreements, and the minimum infrastructure to start.

Activities

- Confirm sponsor, product owner, engineering lead, compliance lead, and design lead.
- Agree decision rights and a weekly risk review.
- Set up repositories, environments, CI, ticketing, and documentation space.
- Draft the initial risk register and assumption log.
- Schedule user research and legal engagement.

Deliverables: project charter, RACI, communication plan, initial backlog, and environment access for all members.

Exit criteria: charter approved, team onboarded, budget confirmed.

10.3 Phase 1: Discovery and feasibility (weeks 3 to 8)

Objectives: reduce the biggest unknowns before committing to architecture.

Activities

- Conduct 15 to 20 user interviews across the four personas and analyze competing tools.
- Define metrics baselines and the beta success thresholds.
- Run the aggregator bake-off with real test accounts; record coverage, reliability, and cost.
- Run LLM feasibility spikes: can the tool-based design answer the top 50 questions accurately and within latency targets?
- Prototype the calculation engine for budgets, goals, and debt payoff, and validate against independent references.
- Prototype categorization on a sample dataset to estimate achievable accuracy.
- Engage counsel; obtain a written view on product classification and disclosure needs.
- Complete a first threat model and privacy impact assessment.
- Build a clickable prototype and test it with at least 10 users.
- Estimate cost per active user across model, aggregator, and infrastructure.

Deliverables: scope document, vendor decisions, legal opinion, feasibility report with go or no-go recommendation, unit economics model, and a refined plan and risk register.

Exit criteria: an aggregator selected with acceptable coverage; grounded answer accuracy of at least 95% on the spike question set with a credible path to 100% for numerical claims; legal path confirmed; cost per active user within budget; prototype tested positively with users.

10.4 Phase 2: Foundation and design (weeks 9 to 14)

Objectives: build the skeleton and the guardrails before the features.

Activities

- Finalize architecture decision records and deployment topology.
- Implement authentication, consent management, and the connector abstraction.
- Establish the financial data model and ingestion pipeline with idempotency and reconciliation.
- Build the LLM gateway, orchestrator skeleton, and tool contract framework.
- Stand up the evaluation harness with the first 100 scenarios and automated runs in CI.
- Establish the design system, conversation patterns, tone guidelines, and accessibility checklist.
- Implement observability: logs, traces, dashboards, and alerts.
- Define the policy rule set with counsel and the compliance lead.
- Set up security tooling and begin recurring dependency and code scans.

Deliverables: running walking skeleton deployed to a staging environment, evaluation harness, design system, policy rule set v1, and updated estimates.

Exit criteria: a user can sign up, connect a test account, and ask one grounded question end to end in staging; evaluation runs automatically; security baseline checks pass; design approved by test users.

10.5 Phase 3: Core build (weeks 15 to 28)

The build is organized as a thin end-to-end slice first, then feature increments in two-week cycles. Each increment ships something usable internally.

Increment A (weeks 15 to 18): Spending insights

- Transaction categorization v1, user corrections, and spending summaries.
- Conversational questions about spending with provenance.

Increment B (weeks 19 to 22): Budgets and goals

- Budget creation, suggestions, and tracking.
- Goal creation, projections, and progress views.

Increment C (weeks 23 to 26): Debt, forecasting, and alerts

- Debt payoff planner and strategy comparison.
- Cash flow forecasting with ranges.
- Proactive notifications and preference controls.

Increment D (weeks 27 to 28): Trust and control features

- Data export, deletion, and disconnect flows.
- Feedback capture, admin console, and audit search.

Parallel tracks throughout: evaluation set growth, policy rule refinement, performance tuning, accessibility testing, and internal dogfooding with at least 30 employees or friendly users.

Exit criteria: all must-have requirements implemented; evaluation targets met on accuracy, groundedness, and policy compliance; no open critical or high severity defects; internal users report the bot as useful in a structured survey.

10.6 Phase 4: Hardening and closed beta (weeks 29 to 36)

Objectives: prove safety, security, and value with real users before opening the doors.

Activities

- Independent penetration test and remediation; privacy and compliance audit.
- Load and failure testing, including aggregator and model provider outage drills.
- Red-team exercises focused on prompt injection, data leakage, and harmful advice.
- Closed beta with 300 to 500 users recruited across the personas, with in-app surveys and weekly review of sampled transcripts.
- Tune categorization, forecasts, notifications, and tone from beta feedback.
- Finalize support playbooks, incident runbooks, and the kill switch procedure.

Exit criteria: zero open critical or high security findings; accuracy on numerical claims at 100% in regression; policy violation rate below the target; beta helpfulness at or above 85%; activation and retention within range of targets; legal sign-off on final copy and disclosures.

10.7 Phase 5: Launch and stabilization (weeks 37 to 40 and beyond)

Objectives: release safely and learn quickly.

Activities

- Staged rollout: 5%, 25%, 50%, then 100% of new signups, with automatic pause conditions.
- Daily metrics and safety review for the first two weeks.
- Support staffing for launch volume; rapid-fix process for categorization and data issues.
- Post-launch retrospective and roadmap planning for release 2.

Exit criteria: two consecutive weeks within reliability and safety targets at full rollout; support backlog stable; sponsor accepts handover to steady-state operations.

11. Timeline, Team, and Budget

11.1 Milestones

Milestone	Target week
Charter approved	2
Go or no-go on feasibility	8
Walking skeleton in staging	14
Internal alpha (increments A and B)	22
Feature complete	28
Closed beta start	31
Penetration test complete and remediated	34
General availability decision	36
100% rollout	40

11.2 Team

Role	Count	Notes
Product manager	1	Owns scope and metrics
Design (product and conversation)	1 to 2	Includes content design for tone
Engineering lead	1	Architecture and delivery
Backend engineers	3	Data, orchestration, calculation engine
Mobile and web engineers	2 to 3	Cross-platform
Data and ML engineer	1 to 2	Categorization, evaluation
QA and test automation	1	Includes evaluation scenarios
Security and privacy engineer	1	May be shared early on
Compliance and legal advisor	0.3 to 0.5	External counsel plus internal lead
Support and operations lead	0.5	Joins from Phase 4

Key-person risk is real in a team this size. Ensure at least two people understand the calculation engine, the aggregator connector, and the policy layer.

11.3 Budget structure

Rather than quote figures that depend on location and vendors, this report recommends tracking cost in five buckets: people (usually 65 to 75% of total), cloud and tooling, third-party services (aggregator, model usage, identity, messaging), external assurance (penetration testing, legal, audit), and contingency of 15 to 20%. Establish **cost per monthly active user** in Phase 1 and monitor it throughout; model usage and aggregator fees scale with engagement and can erode margins if left unchecked. Cost controls include caching, routing simple questions to smaller models, limiting refresh frequency, and capping tool calls per conversation.

12. Dependencies

Dependency	Type	Needed by	Owner	Mitigation if delayed
Aggregator contract and sandbox access	External	Week 4	Product and procurement	Run two vendors in parallel; use mock connector and CSV import
LLM provider agreement with data terms	External	Week 4	Engineering lead and legal	Qualify a second provider; prototype with synthetic data
Legal opinion on classification	External	Week 8	Compliance lead	Constrain scope to educational content; delay any gray-area features

Dependency	Type	Needed by	Owner	Mitigation if delayed
Cloud environment meeting security baseline	Internal	Week 6	Platform and security	Use pre-approved landing zone; reduce custom infrastructure
Identity and consent flows approved	Internal	Week 12	Security and legal	Start with minimal consent model; iterate
Design system and tone guidelines	Internal	Week 12	Design lead	Adopt an existing component library
Labeled categorization dataset	Internal	Week 12	Data and ML	Label a sample in-house; use user corrections in beta
Beta user recruitment	Internal	Week 28	Product and marketing	Start waitlist at Phase 3 midpoint
App store review and policies	External	Week 30	Mobile lead	Submit early builds for review; prepare privacy labels
Penetration test vendor booking	External	Week 30	Security lead	Book in Phase 2; hold slot
Support tooling and staffing	Internal	Week 34	Operations	Founders or engineers cover launch; contract support

12.1 Critical path

The critical path runs through: aggregator selection, then ingestion and data model, then categorization quality, then insight and goal features, then evaluation targets and security remediation, then beta, then launch. The legal opinion is a parallel gate that can reshape scope; treat a late opinion as a critical-path delay.

12.2 Sequencing rules

1. Data model before tool contracts; tool contracts before prompts.
2. Calculation engine and its tests before any conversational feature that uses it.
3. Evaluation harness before behavior tuning.
4. Consent and deletion flows before any real user data in staging.
5. Policy rule set before external beta.

13. Risk Register

Likelihood and impact are rated Low, Medium, or High. Owners are roles, not individuals.

#	Risk	L	I	Mitigation	Early warning trigger	Owner
1	Bot gives incorrect or misleading	M	High	Numbers only from tools; grounding checks;	Any grounding failure in regression or	Engineering lead

#	Risk	L	I	Mitigation	Early warning trigger	Owner
	financial guidance			sampled human review; kill switch	production sampling	
2	Product is classified as regulated advice	M	High	Early legal opinion; scope limits; approved language; no securities recommendations	Counsel flags feature as gray area	Compliance lead
3	Aggregator coverage or reliability falls short	M	High	Bake-off; connector abstraction; manual and file import fallback	Connection success below 90% in testing	Product manager
4	Security breach or data leak	L	High	Threat modeling; minimization; penetration tests; LLM-specific defenses; incident drills	Critical finding in testing; anomalous access logs	Security lead
5	Prompt injection or cross-user data exposure	M	High	Isolation by design; tool permission scoping; red-teaming	Any cross-context leakage in tests	Security and ML
6	Categorization accuracy too low, undermining insights	H	Med	Layered approach; user corrections; benchmark with release gate	Accuracy below 90% on benchmark	Data and ML
7	LLM cost or latency exceeds targets	M	Med	Caching; model routing; token budgets; load tests	Cost per active user above plan by 20%	Engineering lead
8	Model provider outage, change, or deprecation	M	Med	Gateway; pinned versions; tested fallback; regression on candidate models	Provider deprecation notice or elevated errors	Engineering lead
9	Users do not trust or adopt the bot	M	High	Provenance for every number; transparency; early user testing; empathetic tone	Activation or helpfulness below target in alpha	Product manager
10	Scope creep, especially	H	Med	Scope rules; phase gate reviews; metric-linked changes	Unplanned work above 15% of sprint	Product manager

#	Risk	L	I	Mitigation	Early warning trigger	Owner
	investment features					
11	Key-person dependency or hiring delay	M	Med	Pairing; documentation; early hiring; contractor bench	Open role beyond four weeks	Engineering lead
12	Regulatory or privacy law changes in the launch market	L	Med	Counsel monitoring; configurable retention and consent; modular compliance layer	New guidance or enforcement action	Compliance lead
13	Vulnerable users receive inappropriate responses	M	High	Distress detection; approved handling; reviewer escalation; safe defaults	Missed detection in red-team or review	Product and compliance
14	App store rejection or delay	L	Med	Early submissions; policy review; privacy labels ready	Reviewer questions on data use	Mobile lead
15	Vendor lock-in or unfavorable commercial terms	M	Low	Abstraction layers; negotiated exit and data portability	Price changes or restrictive terms	Procurement

13.1 Reading the register

Risks 1, 2, 4, 5, 9, and 13 define the project's quality bar. They should be reviewed weekly and discussed at every gate. If any rises to high likelihood and high impact, the response should be scope reduction or delay, not hope.

13.2 Risk response practices

- Convert the riskiest assumptions into time-boxed spikes in Phase 1 with explicit pass criteria.
- Hold a pre-mortem at each gate: assume the project failed and list the most plausible reasons.
- Track residual risk after mitigation and record decisions to accept risk, with the accepting owner named.
- Keep contingency (time and budget) unspent until a named risk materializes.

14. Testing and Quality Strategy

Testing follows the pyramid with additions specific to AI products.

- **Calculation engine:** property-based and example-based tests, cross-checked against independent implementations and spreadsheets; every formula documented with assumptions.
- **Data pipeline:** replay tests with recorded provider payloads, idempotency checks, and reconciliation of balances against transaction sums.
- **API and integration:** contract tests for tools and connectors; sandbox tests against the aggregator.

- **Conversation quality:** the evaluation layers from Section 8.6, run in CI and before each release.
- **Security:** SAST, DAST, dependency scanning, secrets scanning, threat model reviews, penetration test.
- **Performance and resilience:** load tests, chaos tests for dependency failures, timeouts and retry behavior.
- **Usability and accessibility:** moderated tests, screen reader audits, and contrast checks.
- **Release readiness:** a go-live checklist covering monitoring, runbooks, support readiness, legal sign-off, and rollback.

Definition of done for any feature: requirements met, tests automated, evaluation scenarios added, privacy and security review complete, accessibility checked, documentation updated, and analytics events verified.

15. Rollout, Operations, and Support

15.1 Rollout controls

Use feature flags and staged cohorts. Define automatic pause conditions in advance: for example, grounding failures above threshold, error rate above 2%, or a spike in negative feedback. Maintain a one-step rollback for clients and backend, and a safe mode in which the bot falls back to dashboards and approved static guidance if the model layer misbehaves.

15.2 Monitoring

Area	Signals
Reliability	Availability, latency percentiles, error rates, queue depth
Data health	Connection success rate, data freshness, ingestion lag, duplicate rate
AI quality	Grounding failures, refusal rate, helpfulness ratings, tool error rate, regeneration rate
Safety	Policy violations, distress detections, escalations, reviewer findings
Cost	Tokens per conversation, cost per active user, aggregator calls per user
Product	Activation, retention, goal progress, notification opt-outs

15.3 Support model

Offer tiered support: in-app help and a searchable knowledge base, then a ticket queue with defined response targets, then engineering escalation. Support agents see masked data by default and request just-in-time access with an audit trail for sensitive cases. Maintain playbooks for broken connections, incorrect categorization, disputed answers, data deletion requests, and suspected fraud.

15.4 Continuous improvement cadence

Weekly: review sampled transcripts, top failure cases, and metric movements. Monthly: refresh the evaluation set, review costs, and review risks. Quarterly: re-evaluate vendors, review the regulatory landscape, and revisit roadmap and scope for the next release.

16. Assumptions, Open Questions, and Next Steps

16.1 Open questions that most affect the plan

1. **Which market and regulatory regime does the bot launch in first?** This sets the aggregation options, consent model, and legal opinion.
2. **Is the product free, subscription, or sponsored?** Monetization affects conflict-of-interest analysis and the payment scope.
3. **Will it be a standalone app or embedded in an existing bank or platform?** Embedding changes identity, data access, and compliance responsibilities.
4. **What is the true timeline and team size?** The plan flexes by cutting scope, not by compressing safety and testing.
5. **Is investment guidance required at launch?** If so, add a regulatory workstream and expect several additional months.
6. **Are there data residency or model hosting constraints?** These may require regional deployments or specific providers.
7. **What is the acceptable cost per active user?** This drives model and aggregator choices.

16.2 Recommended next steps

1. Confirm the open questions above and replace the planning assumptions in Section 2.4.
2. Appoint the product owner, engineering lead, and compliance lead, and approve the charter.
3. Start the aggregator bake-off and legal engagement in parallel in week 1.
4. Build the calculation engine prototype and the first 50 evaluation scenarios as the earliest technical proof points.
5. Schedule the first phase gate review for week 8 with the exit criteria in Section 10.3.

Appendix A: Glossary

Term	Meaning
Aggregator	A service that connects to financial institutions, with user consent, to retrieve account and transaction data
Grounding	Requiring that statements in a response are supported by retrieved data or tool output
Prompt injection	An attack that embeds instructions in user or retrieved content to manipulate the model
Tool call	A structured request from the model to a deterministic function
Provenance	The traceable origin of a number or statement, such as the source transactions and formula
Walking skeleton	A minimal end-to-end implementation that exercises the main architectural components
Kill switch	A control that quickly disables a feature or reverts to a safe fallback
Cost per active user	Total variable cost divided by monthly active users

Appendix B: Phase Gate Checklist

- Exit criteria for the phase are met and evidenced.
- Top risks reviewed; new risks added; residual risk accepted by named owners.
- Budget and schedule variance within tolerance, or a re-plan approved.
- Security, privacy, and compliance review items closed or scheduled.
- Open decisions recorded with owners and dates.
- Go, adjust, or stop decision recorded by the sponsor.
-